A Summary and Analysis of Various Black-Box Adversarial Attack Techniques

Connor Hatfield and Nandhu Jani

Introduction

Within the field of adversarial attacks, there exists two different categories: white-box and black-box. White-box adversarial attacks are attacks which know how the target model works (namely, the target model's inputs, outputs, training data, and even its parameters). While white-box attacks can often quickly generate adversarial examples due to its greater understanding of the system it is trying to fool, such cases are rare in the real world, where often such things are hidden away/inaccessible to those performing the attacks.

To better simulate these real-world conditions is where black-box adversarial attacks are of use - usually, only the inputs, classification probabilities, and outputs of the system are known, and sometimes even less. In addition, these observables are sometimes limited in their own right, so black-box attacks often aim to generate perturbed inputs with as few queries as possible, and aim to be *transferable* (that is, designed in such a way that its attacks work on other similar models). In this paper, we synthesize various sources (most notably Bhambri and Muku, et al.) in order to present some of the multitudes of methods and techniques employed in black-box adversarial attack generation so as to better educate those interested in this burgeoning field.

Black-Box Adversarial Attack Techniques

I. Summary

1 - ZOO

One method of performing a black-box adversarial attack is via *gradient estimation*, wherein the gradients of a DNN are determined by evaluating function values at two close points (such as $x+hv$ and $x-hv$), after which the usual gradient descent techniques are used to extrapolate on the estimated gradients.

The above forms the foundation of ZOO (Zeroth Order Optimization) technique suggested by Chen et al., wherein a naive solution for the gradient is found via symmetric difference quotient, namely

$$\hat{g} := \frac{\partial f(x)}{\partial x_i} \approx \frac{f(x + he_i) - f(x - he_i)}{2h},$$

where $\hat{g}$ is the gradient, *f(x)* is the probability scores of the model given the input data point x, h is a small constant (usually around 0.0001), and e_i is a standard basis vector (where all components equal 0 except for one component which is equal to 1).

Something which the paper notes is that determining such a solution would require at least *2p* queries, where p is equal to the dimensionality of the input, and in turn provides two variants, ZOO-Adam and ZOO-Newton, which user their respective equations to improve querying performance.

2 - Opt-Attack

ZOO is a versatile method, but depends on the adversarial attacker being able to access the probability scores of the target model in order to formulate its attack; this is not always possible in the real world. Thus, Cheng et al. modifies the ZOO methodology in order to work with scenarios where querying the target model results only in information about the target label and name it the Opt-Attack technique.

In Opt-Attack, the search direction θ is provided to calculate the distance $g(\theta)$ from an input image x_0 to its closest adversarial example via the below objective functions depending on the type of attack:

$$UntargetedAttack: \quad g(\theta) = min_{\lambda>0}\, \lambda \;\; s.t \;\; f(x_0 + \lambda * \frac{\upsilon}{||\theta||}) \neq y_0$$

$$TargetedAttack(giventargett): \quad g(\theta) = min_{\lambda>0}\, \lambda \;\; s.t \;\; f(x_0 + \lambda * \frac{\theta}{||\theta||}) = t$$

In order to find $argmin_\theta \; g(\theta)$, Cheng et al. utilizes an iterative version of ZOO where the increment value for each iteration is randomly sampled from a Gaussian distribution.

3 - Query Reduction Using Finite Differences

For instances where there exists no way to access information regarding the target model's gradient or loss function, Query Reduction Using Finite Differences can be used. In this technique, logit loss is calculated using the equation

$$l(x, y) = \phi(x+\delta)_y \; - \; max\,\{\phi(x+\delta)_i \;:\, i \neq y\}$$

,

where y is the true label for an input data point x, and $\phi(x+\delta)$ is the logit in question; from these logit values and resulting SoftMax probabilities, a constant to be canceled out can be determined. From this, the difference between the logits of y and those of the next most likely label y' can be found by the adversarial attacker, but at the cost of $O(d)$ queries per input, where d is the input's dimensionality. In order to combat this, Bhagoji et al. suggests using either randomly selected feature group or a PCA for gradient estimation.

4 - Attack Using Gray Images

Sometimes in the real world, the adversarial attacker is unable to obtain any natural inputs for the purpose of feeding into the target model (and thus obtaining by extension its output and/or class probabilities). For these cases, Du & Fang et al. used a new technique, Gray Images, wherein adversarial samples were initialized from gray images of target model inputs (i.e. the pixel values were set between a normalized pixel distribution of [0, 1]). In this way, the adversarial model perturbs the sample image by modifying pixel values to either 1 (making it brighter) or 0 (making it darker), maximizing the probability term $P(y|\theta)$ in the fitness function

$$J(\theta) = P(y'|\theta) \approx [F(\theta)]_{y'} ,$$

until the target model incorrectly classifies the perturbed input θ as some other class y'. Since this is a region-based attack method, gradient estimations depend on what the region space is, which can be done via projected gradient ascent.

5 - AutoZOOM

AutoZOOM (which stands for "Autoencoder-based Zeroth Order Optimization Method") is a black-box adversarial algorithm based on ZOO and aimed at reducing (1) target model querying and (2) attack generation time. This method, written of by Tu et al., utilizes an adaptive random gradient estimation method for (1) and an autoencoder trained offline on unlabelled data for (2).

The optimization problem AutoZOOM seeks to solve is:

$$min_{x \in [0,1]^d} Dist(x, x_0) + \lambda . Loss(x, M(F(x)), t) ,$$

where $Dist(x, x_0)$ is the p-norm, $F(x) : [0, 1]^d \rightarrow \mathcal{R}^k$, $M(.)$ is the monotonic function applied on classifier outputs, $Loss(.)$ is the objective of the attack, reflecting the probability of predicting the argmax k of $[M(F(x))]_k$, and λ is the regularization constant. In addition, the Symmetric Different Quotient Method from the ZOO Method is scaled to become a full gradient estimator, controlling the error by taking an average over q random directions. Using this method, Tu et al. also shows that query efficiency can be improved by adding perturbations to lower dimensions and iterating this process.

6 - Bandits and Priors

Another example of using gradient estimation in order to generate adversarial images in the Bandit and Priors model, which as stated in its name, uses bandit optimization along with prior information on the gradient. As a preface, bandit optimization (also known as Multi-Armed Bandit) is a paradigm in machine learning consisting of an agent using its current state/context along with prior information in order to make the decision upon which its reward is based upon; this framework relies on the agent collecting information on "unsuccessful" attempts in order to minimize initial fluke results when analyzing its decision space.

Priors in this technique are broken down into two categories, time-dependent priors and data-dependent priors. In addition, Ilyas et al. notes that even the known input data points for the target model are predicated on being of a predictable structure, something which can be exploited in order to better understand the relationship between a target model's gradient with respect to a collection of close coordinates. In this way, untargeted adversarial attacks can be performed with both L_2- and L_∞-norm based models considered.

7 - Local Substitute Model

The concept behind a Local Substitute Model is relatively simple, as explained in Papernot & McDaniel et al.: the target model *O* (for oracle) is queried on synthetically-crafted input data to generate its labels, both of which form the dataset upon which an approximation of the target model, *F*, is created. The focus here is on transferability, in that if the adversarial attack is able to fool the approximated model *F*, it will be able to do the same for the target model *O*.

This approach, while effective, suffers under the constraints that in a black-box attack, 1) the architecture of *O* might not be known to the attacker, and 2) the number of queries the attacker can make to *O* may be limited; both of these things lead to a less-accurate approximation *F*, which in turn means less transferability. To address these things, Papernot & McDaniel et al. suggest Jacobian-based Dataset Augmentation, wherein a new set of synthetic data points, S_{p+1}, is generate from the previous training set, S_p, upon which the target model is approximated via the equation

$$S_{p+1} = \{\vec{x} + \lambda \cdot sign(J_F[\check{O}(\vec{x})] : \vec{x} \in S_p)\} \cup S_p\ ,$$

where J_F is the DNN's Jacobian matrix evaluated at several input points $\vec{x}$, $\check{O}$ is the target model/oracle, and λ is the augmentation step size. Of note is that according to the paper, the number, size, and type of layers of the approximation model *F* does not matter in whether an adversarial attack is successful or not, broadening the possibilities for this method.

8 - Gradient Free

The Gradient Free method is one of the more relatively simple local search methods to create adversarial attacks. It starts by randomly selecting a pixel from the image and adding perturbations to it. Once that is done, the image is sent through the target DNN and the classification accuracy is noted. Then, another pixel randomly chosen from a square of size *p* (of which is a hyperparameter) and then the image is sent through the DNN again. This process is repeated however many times the attacker desires and during this process, the change in classification accuracy can help determine which pixels are the most important and which perturbations are the most impactful. This process provides us with two important results. The first being a (hopefully) successful adversarial attack on a DNN. The second, more significant result is that during this process a family of images is generated that can be used to attack a network. These images are called neighboring images because each one is only one pixel off from being an exact copy of its predecessor and successor.

The authors of this paper, Narodytska et al., focused on k-misclassification when performing the attack. K-misclassification is an alternative, more difficult goal of adversarial attacks. It is where, instead of simply misclassifying the image, the adversary wants the confidence score of the correct label to not be present in the top *k* results. They also suggested that, due to the naturally high runtime of this process, that slight noise be added at lower level dimensions to help expedite the process.

9 - Boundary Search

In the domain of adversarial attacks, most do so by taking a known input and its true output, and modifying said input until it reaches an "event-horizon" where it is still recognizable as the true output to humans, but is misclassified by the target model. With Boundary Search, an inversion of this process is done after the above in order to fine-tune the adversarial attack.

Once an adversarial sample proven to fool the target model is generated, a series of further perturbations are made to the adversarial input in order to close the distance between it and the unperturbed, original image in L2 space via a series of random orthogonal steps paired with a closing of distance step (both of which can be altered via hyperparameters). This continues until the target model once again starts identifying the adversarial input with the true output label, at which point the previous image is used. Thus, as stated in Brendel and Rauber et al., the "boundary" of classification is scoped out by the adversarial attack model such that the adversarial sample is more adversarial (more likely to pass as the true output by humans while still being wrongly classified by the model).

This sort of attack method is meant to focus on DNNs (such as ImageNet), where its very nature renders it a "black box" of sorts; thus, finding the DNN's boundaries for classification can prove to be very informative to those attempting to study it.

10 - SimBA

As outlined in Guo & Gardner et al., SimBA (standing for Simple Black-Box Attacks) is a method for black-box adversarial attacks wherein the perturbed vector is generated using the equation

$$x_{adv} = x \pm q\varepsilon,$$

where x is the original input, ε is the step size, and q is a single vector from a set of orthonormal vectors Q. $q\varepsilon$ is first added to see if P(y | x) decreases, and if not, it is subtracted. So as to reduce the number of queries, Guo & Gardner et al. suggest that q only be selected if it does not have two directions which cancel each other out; interestingly, a relatively steep ε will almost always result in a negative change to the probability, leaving its choice of value dependent only on budget considerations.

For determining what constitutes the set of orthonormal vectors Q, both a Cartesian basis and discrete cosine basis are considered: a Cartesian basis would translate to pixel space, i.e. changing the values of a randomly selected pixel in an L_0-attack, whereas a discrete cosine basis would consist of random noise in a low-frequency space via 2D signal mapping of cosine wave magnitude coefficients. However, Guo & Gardner et al. assert a general basis where any orthonormal basis will work given efficient sampling.

II. Analysis of Technique Performance

Three data sets were used to analyze the performance of each method: ImageNet, MNIST, and CIFAR-10. Each data set had a portion of samples that were targeted attacks and a portion that were untargeted attacks. The tables below show the applicable results of each method.

Method	Model	Average # Queries	Average L_2	Attack Success Rate	Specific Parameters Used
Chen et al. [10]	InceptionV3	-	1.19916	88.90%	Step Size = 0.002, # Iterations = 1500
Brendel et al. [11]	ResNet-50	192102	4.876	100%	-
Cheng et al. [13]	ResNet-50	145176	4.934	100%	-
Moon et al. [22]	InceptionV3	722	-	98.50%	L_∞ setting, ϵ = 0.05, Max # Queries = 10,000
Guo et al. [25]	InceptionV3	1283	3.06	97.80%	Step Size = 0.2, # Iterations = 10,000

Table 2. Comparison of methods used for generating an untargeted attack on the ImageNet data set.

Method	Model	Average # Queries	Average L_2	Attack Success Rate	Specific Parameters Used
Chen et al. [10]	InceptionV3	-	3.425	-	Batch Size = 128, # Iterations = 20,000
Ilyas et al. [12]	InceptionV3	11550	-	99.20%	L_∞ setting, ϵ = 0.05
Du et al. [15]	InceptionV3	1701	24.323	100%	Image size-64x64, Momentum factor = 0.4
Tu et al. [16]	InceptionV3	13525	0.000136	100%	Regularization coefficient = 10
Chen et al. [19]	InceptionV3	3573	4.559451	95%	# Iterations = 400
	ResNet-50	3614	3.0575142	98%	# Iterations = 400
	VGG19	3786	4.023045	96%	# Iterations = 400
Brunner et al. [20]	InceptionV3	-	-	85%	Max # Queries = 15,000, L_2 setting, ϵ = 25.89
Moon et al. [22]	InceptionV3	7485	-	99.90%	L_∞ setting, ϵ = 0.05, Max # Queries = 100,000
Ilyas et al. [23]	InceptionV3	1858	-	84.50%	Max # Queries = 10,000, L_2 setting
	InceptionV3	1117	-	95.40%	Max # Queries = 10,000, L_∞ setting
	ResNet-50	993	-	90.30%	Max # Queries = 10,000, L_2 setting
	ResNet-50	722	-	96.60%	Max # Queries = 10,000, L_∞ setting
	VGG16	594	-	82.80%	Max # Queries = 10,000, L_2 setting
	VGG16	370	-	91.60%	Max # Queries = 10,000, L_∞ setting
Alzantot et al. [24]	InceptionV3	11081	61.68669	100%	Max # Queries = 1,000,000, L_∞ setting, ϵ = 0.05
Guo et al. [25]	InceptionV3	7899	9.53	100%	Step Size = 0.2, # Iterations = 30,000

Table 3. Comparison of methods used for generating a targeted attack on the ImageNet data set.

Method	Model	Average # Queries	Average L_2	Attack Success Rate	Specific Parameters Used
Chen et al. [10]	-	-	1.4955	100%	Batch Size = 128, Step Size = 0.01, # Iterations = 3000
Brendel et al. [11]	ResNet-50	131972	1.11545	100%	-
Cheng et al. [13]	ResNet-50	67036	1.0827	100%	-
Bhagoji et al. [14]	-	-	2.7	99.7	Batch Size = 100

Table 4. Comparison of methods used for generating an untargeted attack on the MNIST data set.

Method	Model	Average # Queries	Average L_2	Attack Success Rate	Specific Parameters Used
Chen et al. [10]	-	-	1.9871	98.90%	Batch Size = 128, Step Size = 0.01, # Iterations = 3000
Brendel et al. [11]	ResNet-50	93543	2.0626	100%	-
Cheng et al. [13]	ResNet-50	59094	1.7793	100%	-
Bhagoji et al. [14]	Self-designed, accuracy = 99.2%	62720	-	100%	Batch Size = 100, L_∞ setting, ϵ = 0.3
Tu et al. [16]	C&W [32]	2428	3.55936	100%	Regularization coefficient = 0.1
	C&W [32]	730	3.23792	100%	Regularization coefficient = 1
	C&W [32]	510	3.66128	100%	Regularization coefficient = 10
Chen et al. [19]	-	423	2.352	100%	# Iterations = 100
Alzantot et al. [24]	-	996	-	100%	Step Size = 1.0, Max # Queries = 100,000

Table 5. Comparison of methods used for generating a targeted attack on the MNIST data set.

Method	Model	Average # Queries	Average L_2	Attack Success Rate	Specific Parameters Used
Chen et al. [10]	-	-	0.19973	100%	Batch Size = 128, Step Size = 0.01, # Iterations = 1000
Brendel et al. [11]	ResNet-50	384204	4.8758	-	-
Cheng et al. [13]	ResNet-50	145176	4.9339	-	-
Bhagoji et al. [14]	ResNet-32	-	-	100%	Batch Size = 100
Moon et al. [22]	ResNet-32	1261	-	48%	Max # Queries = 20,000

Table 6. Comparison of methods used for generating an untargeted attack on the CIFAR-10 data set.

Method	Model	Average # Queries	Average L_2	Attack Success Rate	Specific Parameters Used
Chen et al. [10]	same as 14	-	0.39879	96.80%	Batch Size = 128, Step Size = 0.01, # Iterations = 1000
Brendel et al. [11]	ResNet-50	170972	0.2395	-	-
Cheng et al. [13]	ResNet-50	130020	0.2476	-	-
Bhagoji et al. [14]	ResNet-32	61440	-	100%	Batch Size = 100, L_∞ setting, ϵ = 8
Tu et al. [16]	C&W [32]	1524	3.6864	100%	Regularization coefficient = 0.1
	C&W [32]	332	0.00101	100%	Regularization coefficient = 1
	C&W [32]	259	0.00115	100%	Regularization coefficient = 10
Chen et al. [19]	same as 4	381	0.2088	100%	# Iterations = 100
Alzantot et al. [24]		804	-	96.50%	Step Size = 1.0, Max # Queries = 100,000

Table 7. Comparison of methods used for generating a targeted attack on the CIFAR-10 data set.

The two main results that should be focused on are the Attack Success Rate and Average # of Queries with the Average L_2 Distance being just under those two. Every table has at least one notable and significant result for its respective experiments.

References

1) Bhambri, Siddhant et al. "A Survey of Black-Box Adversarial Attacks on Computer Vision Models." (2019).
2) Pin-Yu Chen, Huan Zhang, Yash Sharma, Jinfeng Yi, and Cho-Jui Hsieh. Zoo: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models. In Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security, AISec '17, pages 15–26, New York, NY, USA, 2017. ACM.
3) Minhao Cheng, Thong Le, Pin-Yu Chen, Jinfeng Yi, Huan Zhang, and Cho-Jui Hsieh. Query-efficient hard-label black-box attack: An optimization-based approach. CoRR, abs/1807.04457, 2018.

4) Arjun Nitin Bhagoji, Warren He, Bo Li, and Dawn Song. Practical black-box attacks on deep neural networks using efficient query mechanisms. In The European Conference on Computer Vision (ECCV), September 2018.
5) Yali Du, Meng Fang, Jinfeng Yi, Jun Cheng, and Dacheng Tao. Towards query efficient black-box attacks: An input-free perspective. In Proceedings of the 11th ACM Workshop on Artificial Intelligence and Security, AISec ’18, pages 13–24, New York, NY, USA, 2018. ACM.
6) Chun-Chen Tu, Pai-Shun Ting, Pin-Yu Chen, Sijia Liu, Huan Zhang, Jinfeng Yi, Cho-Jui Hsieh, and Shin-Ming Cheng. Autozoom: Autoencoder-based zeroth order optimization method for attacking black-box neural networks. CoRR, abs/1805.11770, 2018.
7) Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z. Berkay Celik, and Ananthram Swami. Practical black-box attacks against machine learning. In Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security, ASIA CCS ’17, pages 506–519, New York, NY, USA, 2017. ACM.
8) Nina Narodytska and Shiva Prasad Kasiviswanathan. Simple black-box adversarial perturbations for deep networks. CoRR, abs/1612.06299, 2016.
9) Wieland Brendel, Jonas Rauber, and Matthias Bethge. Decision-based adversarial attacks: Reliable attacks against black-box machine learning models. arXiv preprint arXiv:1712.04248, 2017.
10) Andrew Ilyas, Logan Engstrom, and Aleksander Madry. Prior convictions: Black-box adversarial attacks with bandits and priors. CoRR, 07 2018.
11) Chuan Guo, Jacob R. Gardner, Yurong You, Andrew Gordon Wilson, and Kilian Q. Weinberger. Simple black-box adversarial attacks. CoRR, abs/1905.07121, 2019.